```
%% ================================================================
%  motor_predictive_maintenance.m
%  DSP-Based Predictive Maintenance System for Electric Motors
%  B.Tech Final Year Project — VLSI / ECE
% ================================================================
%
%  REQUIRED TOOLBOXES:
%    - Signal Processing Toolbox  (butter, filtfilt, spectrogram, findpeaks)
%    - Statistics & ML Toolbox    (fitcsvm, cvpartition) — OPTIONAL, only
%                                  needed when useSVM = true
%
%  HOW TO RUN:
%    1. Set useRealData = false first (runs on synthetic demo signals).
%    2. Check all 6 figures are generated and saved as PNG files.
%    3. Download CWRU dataset from:
%       https://engineering.case.edu/bearingdatacenter/download-data-file
%    4. Place .mat files in a folder, update dataFolder path below.
%    5. Set useRealData = true and run again.
%
% ================================================================

clear; clc; close all;

%% ----------------------------------------------------------------
%  USER SETTINGS — change these before running
% ----------------------------------------------------------------

useRealData   = false;   % true  = load CWRU .mat files
                         % false = use synthetic demo signals (safe default)

useSVM        = false;   % true  = train SVM (needs labelled data + ML
Toolbox)

doAudio       = false;   % true  = run audio analysis section

% Path to your CWRU dataset folder (only used when useRealData = true)
dataFolder    = 'C:\MotorProject\CWRU';

% Sampling frequency for CWRU 12kHz files
Fs_expected   = 12000;   % Hz

%% ================================================================
%  SECTION 1 — DATA INPUT
% ================================================================
fprintf('=== SECTION 1: DATA INPUT ===\n');

if useRealData
    % ------------------------------------------------------------
    % Load CWRU bearing vibration .mat files
    % ------------------------------------------------------------
```

```
    % CWRU files contain variables named like:
    %   X097_DE_time  — drive-end accelerometer (g), 1-D column vector
    %   X097RPM       — shaft speed (RPM)
    % The exact variable names depend on the file. We search for the
    % first field ending in "_DE_time" automatically.
    % ------------------------------------------------------------

    normalFile = fullfile(dataFolder, 'Normal_0.mat');
    faultFile  = fullfile(dataFolder, '12k_Drive_End_IR007_0_105.mat');

    try
        Sn = load(normalFile);
        Sf = load(faultFile);
    catch ME
        error(['Could not load dataset files.\n' ...
               'Check that dataFolder is correct and files exist.\n' ...
               'Error: %s'], ME.message);
    end

    % Extract drive-end accelerometer field automatically
    fn = fieldnames(Sn);
    de_field_n = fn{contains(fn, 'DE_time', 'IgnoreCase', true)};
    fn2 = fieldnames(Sf);
    de_field_f = fn2{contains(fn2, 'DE_time', 'IgnoreCase', true)};

    normalSig = double(Sn.(de_field_n)(:));
    faultSig  = double(Sf.(de_field_f)(:));
    Fs        = Fs_expected;

    fprintf('Loaded REAL CWRU data.\n');
    fprintf('Normal file : %s\n', normalFile);
    fprintf('Fault file  : %s\n', faultFile);

else
    % ------------------------------------------------------------
    % SYNTHETIC DEMONSTRATION ONLY
    % These signals are NOT real motor recordings.
    % They are generated purely to demonstrate the algorithm flow.
    % Do NOT use synthetic results as project findings.
    % ------------------------------------------------------------
    fprintf('*** SYNTHETIC DEMONSTRATION MODE ***\n');
    fprintf('These signals are NOT real motor data.\n');
    fprintf('Set useRealData=true after downloading the CWRU dataset.\n\n');

    Fs   = 12000;          % Simulate 12 kHz sampling
    t    = (0:1/Fs:2)';    % 2-second signal

    % Normal signal: 50 Hz shaft frequency + light background noise
    normalSig = 0.5*sin(2*pi*50*t) + ...
                0.1*sin(2*pi*100*t) + ...
```

```
                0.05*randn(size(t));

    % Fault signal: add 120 Hz bearing fault frequency + impulsive spikes
    impulseTrain = zeros(size(t));
    impulseTrain(round(linspace(1, length(t), 40))) = 2.5; % periodic impacts
    faultSig  = 0.5*sin(2*pi*50*t) + ...
                0.3*sin(2*pi*120*t) + ...
                0.4*impulseTrain + ...
                0.1*randn(size(t));
end

% --- Choose which signal to analyse (normal or fault) ---
% In the rule-based section we analyse both. Here we set activeSignal.
activeSignal      = faultSig;    % swap to normalSig to test normal case
activeLabel       = 'Fault';     % update if you swap

duration = length(activeSignal) / Fs;
fprintf('\nSignal info:\n');
fprintf('  Sampling frequency : %d Hz\n', Fs);
fprintf('  Signal duration    : %.3f seconds\n', duration);
fprintf('  Total samples      : %d\n', length(activeSignal));

%% ========================================================================
%  SECTION 2 — PRE-PROCESSING
% ========================================================================
fprintf('\n=== SECTION 2: PRE-PROCESSING ===\n');

% --- 2a. DC removal ---
% The mean (DC component) carries no fault information and would
% distort the FFT spectrum if left in.
activeSignal = activeSignal - mean(activeSignal);
normalSig    = normalSig    - mean(normalSig);
faultSig     = faultSig     - mean(faultSig);

% --- 2b. Low-pass Butterworth filter ---
% Cut-off: 5000 Hz for vibration (bearing faults appear below 5 kHz
% at 12 kHz sampling). Filter order 4 gives -80 dB/decade roll-off
% without excessive phase distortion.
% filtfilt applies the filter twice (forward + backward) → zero phase.
fc    = 5000;   % Hz
order = 4;
[b, a] = butter(order, fc / (Fs/2), 'low');

procSignal  = filtfilt(b, a, activeSignal);
procNormal  = filtfilt(b, a, normalSig);
procFault   = filtfilt(b, a, faultSig);

fprintf('DC removed and Butterworth LPF applied (fc = %d Hz, order %d).\n',
fc, order);
```

```
% --- Figure 1: Raw vs Preprocessed ---
Nplot  = min(round(0.05 * Fs), length(activeSignal)); % first 50 ms
tPlot  = (0:Nplot-1) / Fs;

figure('Name','Fig1 Raw vs Preprocessed','NumberTitle','off','Position',[100
100 900 500]);
subplot(2,1,1);
plot(tPlot, activeSignal(1:Nplot), 'b', 'LineWidth', 0.8);
title('Raw Signal (first 50 ms)');
xlabel('Time (s)'); ylabel('Amplitude (g)'); grid on;

subplot(2,1,2);
plot(tPlot, procSignal(1:Nplot), 'r', 'LineWidth', 0.8);
title('Pre-processed Signal (DC removed + LPF)');
xlabel('Time (s)'); ylabel('Amplitude (g)'); grid on;

saveas(gcf, 'fig1_raw_vs_preprocessed.png');
fprintf('Saved: fig1_raw_vs_preprocessed.png\n');

%% ========================================================================
%  SECTION 3 — TIME-DOMAIN ANALYSIS
% ========================================================================
fprintf('\n=== SECTION 3: TIME-DOMAIN FEATURES ===\n');

% Compute for both normal and fault signals
function feat = computeTimeDomain(sig)
    feat.mean        = mean(sig);
    feat.rms         = rms(sig);
    feat.std         = std(sig);
    feat.peak        = max(abs(sig));
    feat.crestFactor = feat.peak / feat.rms;
    feat.kurtosis    = kurtosis(sig);
end

% Note: nested functions won't work in a script in older MATLAB;
% compute inline instead:
computeFeats = @(s) struct( ...
    'meanVal',       mean(s), ...
    'rmsVal',        rms(s), ...
    'stdVal',        std(s), ...
    'peakVal',       max(abs(s)), ...
    'crestFactor',   max(abs(s)) / rms(s), ...
    'kurtVal',       kurtosis(s));

normFeats  = computeFeats(procNormal);
faultFeats = computeFeats(procFault);

fprintf('\n%-20s %-15s %-15s\n', 'Feature', 'Normal', 'Fault');
fprintf('%s\n', repmat('-', 1, 50));
```

```
fprintf('%-20s %-15.5f %-15.5f\n', 'Mean',         normFeats.meanVal,
faultFeats.meanVal);
fprintf('%-20s %-15.5f %-15.5f\n', 'RMS',          normFeats.rmsVal,
faultFeats.rmsVal);
fprintf('%-20s %-15.5f %-15.5f\n', 'Std Dev',      normFeats.stdVal,
faultFeats.stdVal);
fprintf('%-20s %-15.5f %-15.5f\n', 'Peak',         normFeats.peakVal,
faultFeats.peakVal);
fprintf('%-20s %-15.5f %-15.5f\n', 'Crest Factor', normFeats.crestFactor,
faultFeats.crestFactor);
fprintf('%-20s %-15.5f %-15.5f\n', 'Kurtosis',     normFeats.kurtVal,
faultFeats.kurtVal);

%% ======================================================================
%  SECTION 4 — FREQUENCY-DOMAIN ANALYSIS (FFT)
% ======================================================================
fprintf('\n=== SECTION 4: FREQUENCY-DOMAIN ANALYSIS ===\n');

N  = length(procSignal);
Y  = fft(procSignal);
P2 = abs(Y / N);
P1 = P2(1:floor(N/2)+1);
P1(2:end-1) = 2 * P1(2:end-1);  % single-sided correction
f  = Fs * (0:floor(N/2)) / N;

% --- Figure 2: FFT Spectrum ---
figure('Name','Fig2 FFT Spectrum','NumberTitle','off','Position',[100 100
900 400]);
plot(f, P1, 'k', 'LineWidth', 0.8);
title(sprintf('Single-Sided Amplitude Spectrum — %s signal', activeLabel));
xlabel('Frequency (Hz)');
ylabel('Amplitude');
grid on;
xlim([0 Fs/2]);

saveas(gcf, 'fig2_fft_spectrum.png');
fprintf('Saved: fig2_fft_spectrum.png\n');

% Find dominant peaks
minPeakHeight = 0.05 * max(P1);
[peakAmps, peakFreqs] = findpeaks(P1, f, ...
    'MinPeakHeight', minPeakHeight, ...
    'NPeaks', 5, ...
    'SortStr', 'descend');

fprintf('\nTop dominant frequencies:\n');
for k = 1:length(peakAmps)
    fprintf('  Peak %d: %.1f Hz  (amplitude = %.4f)\n', k, peakFreqs(k),
peakAmps(k));
end
```

```
fprintf('Fault-related spikes at non-harmonic frequencies suggest bearing
defects.\n');

%% ======================================================================
%  SECTION 5 — FEATURE EXTRACTION
% ======================================================================
fprintf('\n=== SECTION 5: FEATURE EXTRACTION ===\n');

% Build a fixed-length feature vector (8 elements) for classification.
% Elements: [RMS, StdDev, Peak, CrestFactor, Kurtosis, F1, F2, F3]
% where F1-F3 are the top 3 peak frequencies from FFT.

topFreqs = zeros(1, 3);
nFound   = min(3, length(peakFreqs));
topFreqs(1:nFound) = peakFreqs(1:nFound);

featureVec = [faultFeats.rmsVal, ...
              faultFeats.stdVal, ...
              faultFeats.peakVal, ...
              faultFeats.crestFactor, ...
              faultFeats.kurtVal, ...
              topFreqs];

featureNames =
{'RMS','StdDev','Peak','CrestFact','Kurtosis','DomFreq1','DomFreq2','DomFreq3
'};

fprintf('Feature vector (active signal):\n');
for k = 1:8
    fprintf('  %-12s : %.4f\n', featureNames{k}, featureVec(k));
end

%% ======================================================================
%  SECTION 6 — CONDITION ASSESSMENT
% ======================================================================
fprintf('\n=== SECTION 6: CONDITION ASSESSMENT ===\n');

% ----------------------------------------------------------------------
% METHOD A: Rule-based thresholds
% These threshold values were chosen based on published literature
% guidelines for accelerometer-based bearing monitoring.
% IMPORTANT: For real deployment, calibrate these thresholds from
% your own healthy motor baseline data.
% ----------------------------------------------------------------------
RMS_threshold    = 0.15;   % g  — typical healthy motor RMS < 0.10–0.20 g
Crest_threshold  = 4.5;    % dimensionless — healthy CF typically < 4
Kurtosis_threshold = 4.0;  % healthy kurtosis ≈ 3; > 4 suggests impacts

faultFlags = {};
```

```
if faultFeats.rmsVal      > RMS_threshold,       faultFlags{end+1} = 'High
RMS'; end
if faultFeats.crestFactor > Crest_threshold,      faultFlags{end+1} = 'High
Crest Factor'; end
if faultFeats.kurtVal     > Kurtosis_threshold,   faultFlags{end+1} = 'High
Kurtosis'; end

fprintf('Rule-based thresholds:\n');
fprintf('  RMS threshold    : %.3f g\n',  RMS_threshold);
fprintf('  Crest threshold  : %.2f\n',   Crest_threshold);
fprintf('  Kurtosis threshold: %.2f\n',  Kurtosis_threshold);

if isempty(faultFlags)
    ruleCondition = 'Normal operating pattern';
elseif length(faultFlags) == 1
    ruleCondition = 'Requires further inspection';
else
    ruleCondition = 'Abnormal vibration detected';
end

fprintf('\nFlags raised    : %s\n', strjoin(faultFlags, ', '));
fprintf('Rule-based verdict: [ %s ]\n', ruleCondition);
fprintf('\nNote: This is an algorithmic estimate, not a guaranteed
mechanical diagnosis.\n');

% ------------------------------------------------------------------
% METHOD B: Optional SVM Classifier
% ------------------------------------------------------------------
if useSVM
    fprintf('\n--- SVM CLASSIFICATION ---\n');

    % Build a labelled dataset from CWRU files
    % Each row = feature vector; label = 1 (Normal) or 2 (Fault)
    % You need at least a few files per class. Adjust filenames below.
    cwruFiles = {
        fullfile(dataFolder, 'Normal_0.mat'),    1;
        fullfile(dataFolder, 'Normal_1.mat'),    1;
        fullfile(dataFolder, '12k_Drive_End_IR007_0_105.mat'), 2;
        fullfile(dataFolder, '12k_Drive_End_OR007@6_0_130.mat'), 2;
    };

    X_all = [];
    Y_all = [];

    for fi = 1:size(cwruFiles, 1)
        try
            Stmp  = load(cwruFiles{fi, 1});
            fntmp = fieldnames(Stmp);
            de_f  = fntmp{contains(fntmp, 'DE_time', 'IgnoreCase', true)};
            sig_tmp = double(Stmp.(de_f)(:));
```

```
            sig_tmp = sig_tmp - mean(sig_tmp);
            sig_tmp = filtfilt(b, a, sig_tmp);

            % Extract features
            rms_t    = rms(sig_tmp);
            std_t    = std(sig_tmp);
            peak_t   = max(abs(sig_tmp));
            cf_t     = peak_t / rms_t;
            kurt_t   = kurtosis(sig_tmp);

            N_t = length(sig_tmp);
            Y_t = fft(sig_tmp);
            P_t = abs(Y_t/N_t); P_t = P_t(1:floor(N_t/2)+1);
            P_t(2:end-1) = 2*P_t(2:end-1);
            f_t = Fs*(0:floor(N_t/2))/N_t;
            [~, locs_t] = findpeaks(P_t, f_t, 'NPeaks', 3, ...
                'SortStr','descend','MinPeakHeight',0.01*max(P_t));
            tf = zeros(1,3); tf(1:min(3,length(locs_t))) =
locs_t(1:min(3,length(locs_t)));

            feat_row = [rms_t, std_t, peak_t, cf_t, kurt_t, tf];
            X_all    = [X_all; feat_row];
            Y_all    = [Y_all; cwruFiles{fi, 2}];
        catch
            fprintf('Skipped file (not found): %s\n', cwruFiles{fi,1});
        end
    end

    if size(X_all, 1) < 4
        fprintf('Not enough data loaded for SVM. Add more CWRU files.\n');
    else
        % Train/Test split — 70/30
        cv  = cvpartition(Y_all, 'HoldOut', 0.30);
        X_tr = X_all(cv.training, :);  Y_tr = Y_all(cv.training);
        X_te = X_all(cv.test,     :);  Y_te = Y_all(cv.test);

        % Train SVM with RBF kernel
        svmModel  = fitcsvm(X_tr, Y_tr, 'KernelFunction','rbf',
'Standardize', true);
        Y_pred    = predict(svmModel, X_te);

        fprintf('SVM trained on %d samples, tested on %d samples.\n', ...
            sum(cv.training), sum(cv.test));
        fprintf('Predictions : %s\n', num2str(Y_pred'));
        fprintf('True labels : %s\n', num2str(Y_te'));
    end
end

%% ========================================================================
%  SECTION 7 — GRAPH GENERATION
```

```
% ================================================================
fprintf('\n=== SECTION 7: GRAPH GENERATION ===\n');

% --- Figure 3: RMS Comparison bar chart ---
figure('Name','Fig3 RMS Comparison','NumberTitle','off','Position',[100 100
500 400]);
bar([normFeats.rmsVal, faultFeats.rmsVal], 'FaceColor', [0.2 0.6 0.8]);
set(gca, 'XTickLabel', {'Normal','Fault'});
title('RMS Value Comparison — Normal vs Fault');
xlabel('Motor Condition');
ylabel('RMS Amplitude (g)');
grid on;
text(1, normFeats.rmsVal*1.05, sprintf('%.4f', normFeats.rmsVal),
'HorizontalAlignment','center');
text(2, faultFeats.rmsVal*1.05, sprintf('%.4f', faultFeats.rmsVal),
'HorizontalAlignment','center');
saveas(gcf, 'fig3_rms_comparison.png');
fprintf('Saved: fig3_rms_comparison.png\n');

% --- Figure 4: Spectrogram ---
figure('Name','Fig4 Spectrogram','NumberTitle','off','Position',[100 100 900
400]);
window    = hamming(256);
noverlap  = 200;
nfft      = 512;
spectrogram(procSignal, window, noverlap, nfft, Fs, 'yaxis');
title(sprintf('Spectrogram — %s Signal', activeLabel));
colorbar;
saveas(gcf, 'fig4_spectrogram.png');
fprintf('Saved: fig4_spectrogram.png\n');

% --- Figure 5: Feature comparison bar chart ---
normFV = [normFeats.rmsVal, normFeats.stdVal, normFeats.peakVal, ...
          normFeats.crestFactor, normFeats.kurtVal];
faultFV = [faultFeats.rmsVal, faultFeats.stdVal, faultFeats.peakVal, ...
           faultFeats.crestFactor, faultFeats.kurtVal];
featLabels5 = {'RMS','Std Dev','Peak','Crest Factor','Kurtosis'};

figure('Name','Fig5 Feature Comparison','NumberTitle','off','Position',[100
100 800 400]);
bar([normFV; faultFV]');
legend('Normal','Fault','Location','northwest');
set(gca,'XTickLabel', featLabels5);
title('Feature Comparison — Normal vs Fault Signal');
xlabel('Feature');
ylabel('Feature Value');
grid on;
saveas(gcf, 'fig5_feature_comparison.png');
fprintf('Saved: fig5_feature_comparison.png\n');
```

```
%% ================================================================
%  SECTION 8 — PERFORMANCE EVALUATION
% ================================================================
fprintf('\n=== SECTION 8: PERFORMANCE EVALUATION ===\n');

if useSVM && exist('Y_pred','var') && exist('Y_te','var')

    % Compute confusion matrix values
    TP = sum(Y_pred == 2 & Y_te == 2);
    TN = sum(Y_pred == 1 & Y_te == 1);
    FP = sum(Y_pred == 2 & Y_te == 1);
    FN = sum(Y_pred == 1 & Y_te == 2);

    accuracy  = (TP + TN) / (TP + TN + FP + FN);
    precision = TP / (TP + FP + eps);
    recall    = TP / (TP + FN + eps);
    f1        = 2 * precision * recall / (precision + recall + eps);
    far       = FP / (FP + TN + eps);  % false alarm rate

    fprintf('\nPerformance Metrics:\n');
    fprintf('  Accuracy       : %.2f%%\n', accuracy  * 100);
    fprintf('  Precision      : %.2f%%\n', precision * 100);
    fprintf('  Recall         : %.2f%%\n', recall    * 100);
    fprintf('  F1-Score       : %.4f\n',   f1);
    fprintf('  False Alarm Rate: %.2f%%\n', far      * 100);

    % Confusion chart
    figure('Name','Fig6 Confusion Matrix','NumberTitle','off');
    confusionchart(Y_te, Y_pred, 'ColumnSummary','column-normalized', ...
        'RowSummary','row-normalized');
    title('Confusion Matrix — SVM Classifier');
    saveas(gcf, 'fig6_confusion_matrix.png');
    fprintf('Saved: fig6_confusion_matrix.png\n');

else
    fprintf('SVM evaluation skipped (useSVM=false or insufficient data).\n');
    fprintf('To evaluate: set useSVM=true, ensure CWRU files are
available.\n');

    % Show manual formulas for report
    fprintf('\nPerformance formula reference:\n');
    fprintf('  Accuracy  = (TP+TN)/(TP+TN+FP+FN)\n');
    fprintf('  Precision = TP/(TP+FP)\n');
    fprintf('  Recall    = TP/(TP+FN)\n');
    fprintf('  F1        = 2*Prec*Rec/(Prec+Rec)\n');
    fprintf('  FAR       = FP/(FP+TN)\n');
end

%% ================================================================
%  SECTION 9 — OPTIONAL AUDIO ANALYSIS
```

```
% ================================================================
if doAudio
    fprintf('\n=== SECTION 9: AUDIO ANALYSIS ===\n');

    audioFile = 'motor_sound.wav';  % update path to your WAV file

    if ~isfile(audioFile)
        fprintf('Audio file not found: %s\n', audioFile);
        fprintf('Place a motor .wav recording in the working directory.\n');
    else
        [audioSig, Fa] = audioread(audioFile);
        audioSig = audioSig(:, 1);   % use channel 1 if stereo
        audioSig = audioSig - mean(audioSig);

        fprintf('Audio: Fs = %d Hz, Duration = %.2f s\n', Fa,
length(audioSig)/Fa);

        % Waveform
        figure('Name','Audio Waveform','NumberTitle','off');
        ta = (0:length(audioSig)-1) / Fa;
        plot(ta, audioSig);
        title('Motor Sound Waveform');
        xlabel('Time (s)'); ylabel('Amplitude'); grid on;
        saveas(gcf, 'fig_audio_waveform.png');

        % Spectrogram
        figure('Name','Audio Spectrogram','NumberTitle','off');
        spectrogram(audioSig, hamming(512), 400, 1024, Fa, 'yaxis');
        title('Motor Sound Spectrogram');
        colorbar;
        saveas(gcf, 'fig_audio_spectrogram.png');

        % Basic audio features
        audio_rms = rms(audioSig);
        % Zero-crossing rate
        zcr = sum(abs(diff(sign(audioSig)))) / (2 * length(audioSig));
        % Spectral centroid
        Na = length(audioSig);
        Ya = abs(fft(audioSig));
        Ya = Ya(1:floor(Na/2)+1);
        fa_vec = Fa * (0:floor(Na/2)) / Na;
        sp_centroid = sum(fa_vec' .* Ya) / sum(Ya);

        fprintf('Audio features:\n');
        fprintf('  RMS              : %.5f\n', audio_rms);
        fprintf('  Zero-crossing rate: %.5f\n', zcr);
        fprintf('  Spectral centroid : %.2f Hz\n', sp_centroid);

        fprintf('\nNote: Microphone-based detection is highly sensitive
to\n');
```

```
        fprintf('background noise and motor-to-sensor distance.\n');
        fprintf('Audio analysis supplements — but does not replace —
vibration analysis.\n');
    end
end

%% ================================================================
%  SECTION 10 — HARDWARE & FPGA NOTES (comment reference)
% ================================================================
%
%  HARDWARE EXTENSION (ESP32):
%
% ──────────────────────────────────────────────────────────────────
%  - ESP32 ADC (12-bit, up to ~12 kHz effective rate) reads ADXL345
%    over I2C (400 kHz). Each sample is sent via Serial/Wi-Fi to PC.
%  - On PC side: read serial port in MATLAB using serialport(),
%    buffer 12000 samples (1 second), then run this analysis.
%  - Placement: glue/clamp accelerometer on bearing housing (not frame).
%
%  FPGA VLSI EXTENSION (Future Scope):
%
% ──────────────────────────────────────────────────────────────────
%  - RMS:  Accumulate squares in Q16.16 fixed-point; divide by N;
%          take sqrt via CORDIC (16-cycle latency).
%  - FFT:  Radix-2 DIT Cooley-Tukey; 1024-point needs 10 stages,
%          ~10240 butterfly operations; pipelined at 100 MHz clock.
%  - LPF:  Implement IIR as Direct Form II biquad in Verilog;
%          coefficients stored in ROM; quantise to Q2.14.
%  - Resources (Xilinx Artix-7 estimate):
%          1024-pt FFT ≈ 8000 LUTs, 16 DSP48 blocks, 4 BRAMs.
%  - Advantage: Real-time <10 µs latency vs MATLAB batch ~500 ms.
%
% ================================================================

fprintf('\n=== ALL SECTIONS COMPLETE ===\n');
fprintf('Generated figures:\n');
fprintf('  fig1_raw_vs_preprocessed.png\n');
fprintf('  fig2_fft_spectrum.png\n');
fprintf('  fig3_rms_comparison.png\n');
fprintf('  fig4_spectrogram.png\n');
fprintf('  fig5_feature_comparison.png\n');
if useSVM
    fprintf('  fig6_confusion_matrix.png\n');
end
fprintf('\nSaved result to: analysis_result.mat\n');

% Save workspace output
save('analysis_result.mat', 'featureVec', 'featureNames', ...
```

```
    'normFeats', 'faultFeats', 'ruleCondition', 'Fs');

fprintf('\n--- FINAL VERDICT ---\n');
fprintf('Condition assessed as: [ %s ]\n', ruleCondition);
fprintf('This is an algorithmic estimate. Not a guaranteed mechanical
diagnosis.\n');
```